

Depth, Not Length

A Proposal: Why Coding Agents Alone Aren't Enough,
and How a Structured Context Graph Recovers What Similarity Cannot

Kasyap (storyline) | Brief (review & synthesis)

June 2026 | Working design record

This is the consolidated plan that supersedes the earlier storyline and benchmark notes. It reads in the order: the idea, the storyline, what we test, the benchmarks and metrics, and finally the math, meaning how we propose to solve it.

1. The Idea

The bottleneck isn't context length. It's context depth. Coding agents fail not because they run out of tokens but because flat similarity retrieval cannot recover deep causal chains, and you cannot buy your way out with more tokens, because relevance density falls as the window grows. A structured context graph recovers depth that similarity retrieval provably cannot.

The field spent two years optimizing the wrong axis. The token maxxing wave assumed that spending more tokens buys more capability, and the recent correction (the ROI on tokens discourse) observed that this stopped paying off, but only as an observation, without a mechanism. This proposal supplies the mechanism.

The mechanism is depth. A task has *depth* when the thing that makes its output correct lives several causal hops away from the surface of the prompt, such as a decision made months ago, the constraint it implies, and the code that must honor it. Similarity retrieval recovers the surface and the near hops, then collapses. Adding tokens does not fix this, because the deeper you reach, the more the retrieved window fills with plausible but wrong near misses. The cure is not a bigger window. It is a different retrieval object, a typed graph where a causal link is *followed* rather than *guessed*.

This is deliberately a contrarian correction rather than a parallel claim. It rides the live trend, it is provable, and it lands exactly where the structured context graph is strong. It also tells us, honestly, where the graph does *not* help (shallow, fully specified tasks), which is what separates a research result from a vendor whitepaper.

2. The Storyline

The world. Modern agents operate over horizons far longer than any context window, whether a multi week project, a sprawling codebase, or a long chain of product decisions. At every step the agent must answer one question: of everything that has happened, what matters *right now*?

The failure. The agent needs something decided long ago. Similarity retrieval does not surface it, because the new task does not *resemble* the old decision, since the words moved on even though the dependency did not. So the agent produces output that quietly violates the decision. Tests still pass. The violation is caught later, downstream, and the fix burns several more iterations. This is the rework loop, and it is where the cost actually lives.

The fix. Store entities, relations, and crucially the *rationale* as a typed graph. Retrieval then follows the link to the “why” instead of gambling on resemblance. The agent gets it right the first time, so the rework loop never starts.

The shape of the claim. We make the *capability* claim primary: the structured graph recovers causal context that similarity cannot, so the agent builds the right thing. Token efficiency is the *secondary, supporting* result. It falls out of the capability win, because getting it right the first time is cheaper than getting it wrong and redoing it. We do *not* claim “fewer tokens per call.” Storing and walking the why often costs *more* per retrieval. We claim fewer tokens to *reach a correct result*.

The hero and the theory. The theory is about causal and multi hop context recovery in general. The empirical hero is *decisions*, because a decision is the highest depth instance of that general object, a deep “why” chain, and it is the most economically valuable one. Widen the theory and spotlight decisions in the experiments.

The tension that carries it. Shallow tasks hide the problem. Deep tasks expose it. Everything builds toward the depth at which the world tips over.

3. What We Are Testing

One construct, isolated: **context retrieval quality**, with everything else held fixed. Same coding model, same code search tool, same task, with only the memory changing. Three things are tested, each tied to a claim.

1. **Capability (primary).** Does the agent recover the governing context and produce compliant output? Tested by stripping the spec so the answer must come from external context, not the prompt.
2. **Depth dependence (the signature).** Does the advantage behave the way the math predicts as a function of chain depth d , staying flat for the graph, collapsing for similarity, with a crossover? Tested by sweeping d and matching the measured curve to the predicted shape.
3. **Efficiency (secondary).** Is the total cost to correct lower, despite equal or higher per call tokens? Tested as cost per task that is resolved and ready to merge, not tokens per call.

The honesty move, built into the test design. We register the protocol in advance, meaning the claims, datasets, metrics, and model list, before running anything. We deliberately report the regime where the graph adds *nothing* (shallow, fully specified, $d = 1$). Showing the crossover (the graph helps iff $d \geq 2$) is what makes the wins believable. A paper that looks rigged wins everywhere. A credible paper has a sharp crossover.

The depth axis (the one number that runs through everything)

Definition 1 (Task depth). $d(t)$ is the shortest justification path length, in edges, from the task surface (what is literally in the prompt and at the edit site) to the governing decision (the one whose violation makes the output wrong but still test passing), measured in the task’s decision dependency graph.

So $d=1$ is the spec stated at the edit site. $d=2$ is surface $\rightarrow$ constraint $\rightarrow$ decision, and $d \geq 3$ is the deep regime. Depth is a *dial*, not a subjective label, and it is the *same dial* as spec stripping. Strip the immediate constraint and d goes from one to two. Strip its justification as well and it goes from two to three. *Strip count = causal depth = the math’s d* . On borrowed datasets we read d off their existing event and multi hop graphs so hop count is ground truth. Two annotators trace the path independently, and tasks disagreeing by more than one hop are

dropped or rewritten. This collapses “how do we label depth” and “how do we strip the spec” into a single operation.

4. Benchmarks and Metrics

Datasets, three layers, one construct

Layer	Datasets	Role
Neutral front	LongMemEval, LoCoMo, RULER, multi hop HotpotQA	Prove the gap is real on turf we did not design.
Credibility base	SWE-bench and τ -bench, spec stripped into an external store, with vanilla SWE-bench as the $d=1$ control	A famous base that, once stripped, actually tests context depth.
Hero instrument	The extended dcbench, where we own depth labels and the gotcha decisions	The controlled scalpel and the source of the crossover figure.

The apparent tension between “use SWE-bench” and “do not centralize SWE-bench because it is Brief’s weakness” dissolves once the spec is *stripped*. Stripped SWE-bench tests context recovery, not code search, which is exactly the variable we want. Running *vanilla* SWE-bench and claiming victory is the trap. Running it as the $d=1$ control is the honesty point.

Arms: the ladder is the set of controls

A deliberate ladder, so each rung isolates one property. We go from **no memory** to **full context dump** (the brute force, “just buy tokens” control), to **similarity RAG** (Mem0), to **hierarchical** (MemGPT), to **graph** (GraphRAG), to **temporal graph** (Zep), to **Brief** (a typed graph with stored rationale). To this we add a **random context** negative control, and two isolating baselines, namely **spec only** (planning, no memory) and **context only** (memory, no planning). Together they answer the killer question: is the win the memory, or just any structured planning step? Same model and same code search tool across all arms, so memory is the only variable. Beating the full context dump at lower cost to correct is the result that kills “just buy more tokens.”

The metric

Definition 2 (Resolution efficiency).

$$\eta = \frac{\text{tasks resolved (compliant, ready to merge)}}{\text{total tokens to reach that correct result}} = \frac{P}{c},$$

where P is single attempt success probability and c is tokens per attempt.

This is the “quality of tokens” quantity, and it is *not* the discredited “fewer tokens per call” claim. The link is exact. If you retry until correct, expected attempts = $1/P$, so total cost to correct = c/P , and $\eta = 1/(\text{cost to correct}) = P/c$. The two framings are algebraically the same number. The numerator carries the depth win, and the denominator is total cost including rework, not the size of one prompt.

The one wording discipline. Against the full context dump, the graph wins η partly by injecting fewer tokens (a smaller c). Against similarity RAG it does *not*, because there the graph’s c is larger (for example Mem0 in plain mode against graph mode is roughly 7K against

14K per call). So we never justify the similarity case through the denominator. We justify it through the numerator, and the math (next section) shows the recovery gap in P swamps the constant token penalty in c .

5. The Math, How We Propose to Solve It

The design principle is the “transformer feel”: one simple object that becomes rich by being composed with itself, not by adding machinery. Our object is per hop recovery fidelity, and the whole paper is one composition law.

One object, one law

Definition 3 (Per hop fidelity). f_k is the probability that the k th causal link in a chain survives retrieval. A depth d task is resolved only if the whole chain survives, and under independent hops

$$P(d) = \prod_{k=1}^d f_k.$$

Everything below is what f_k looks like in two regimes.

The load bearing step: why $f_k = s_0 \rho^k$ for similarity

This is the single most important argument in the paper, and the entire result rests on it, so it gets the most rigor. The claim is that under similarity retrieval, per hop fidelity decays *geometrically in the hop index* rather than at a constant rate.

The novel mechanism is that **the chain interferes with itself**. To act on a depth d task you must surface node k (the k th link in the why chain) by embedding similarity to the query at node 0. Node k is recovered only if it beats its distractors into the top K . The decisive observation is that the items that most strongly outscore node k are the *shallower nodes of its own chain*. Nodes $1, \dots, k-1$ are each fewer causal hops from the query, so each resembles the query *more* than node k does. They are not random corpus noise. They are high scoring and on topic distractors, and there is exactly one more of them for every hop you go deeper. Modeling each as independently displacing node k with a factor $\rho < 1$, and folding the constant background pool into s_0 ,

$$f_k = s_0 \rho^k.$$

The linear in k exponent is not assumed. It falls out of “the number of chain internal distractors that outrank node k grows by exactly one per hop.” This is precisely *why* remembering the why is structurally hard: the proximate reasons always look more relevant than the root cause, so the root cause is the most buried item exactly when it matters most.

A second, independent mechanism gives the same form. Pure semantic drift means each causal hop reframes the concept, so cosine similarity to the query decays geometrically, and in the linear regime of a softmax or Luce retrieval rule, retrieval probability tracks similarity, again yielding $f_k = s_0 \rho^k$. Two independent causes landing on one expression is the robustness argument, since the result does not hinge on a single modeling choice.

Assumption 1 (where it bends, stated honestly). *Independent interferers and independent hops. Real chains correlate. If a shallow node helps retrieve its neighbor, f_k decays slower than ρ^k and the gap narrows. If intermediate nodes are near duplicates they cluster and one retrieval grabs several, also narrowing it. So ρ^k is the clean independent interference baseline. Correlations shift the exponent but do not flip the quadratic versus linear outcome. This is directly measurable on depth labeled chains, and that measurement is the test of the assumption that we commit to in advance.*

The two regimes

Similarity. The product telescopes:

$$P_{\text{sim}}(d) = \prod_{k=1}^d s_0 \rho^k = s_0^d \rho^{d(d+1)/2} \quad (\text{quadratic exponent}).$$

The clean “ $s_0 = 1$ ” special case, $\rho^{d(d+1)/2}$, is the legible version for the body of a talk, and we keep s_0 in the derivation because it carries honesty about the fact that recovery starts below one.

Structured (graph). A stored typed edge is dereferenced, not searched. Recovering hop k is following a pointer, facing no resemblance competition, failing only if the link was never written or is malformed. Fidelity is flat, $f_k = q$ with $q \approx 1$:

$$P_{\text{graph}}(d) = q^d \quad (\text{linear exponent}).$$

The contrast, the whole paper in one line

$$\frac{P_{\text{graph}}(d)}{P_{\text{sim}}(d)} = \left(\frac{q}{s_0}\right)^d \rho^{-d(d+1)/2} \xrightarrow{d \rightarrow \infty} \infty. \quad \text{Linear exponent over quadratic exponent: the gap does not merely exist, it \textit{widens} with depth, and depth is exactly what remembering “why” demands.}$$

The crossover, charging the graph honestly

On pure recovery the graph leads at every d , so the crossover must be a *cost* statement, which is the honest framing. Let V be the value of a resolved task and W the structured store’s overhead per chain (building and maintaining typed links is real work a flat pool does not pay), and write $\tau = W/V$. Define

$$g(d) = q^d - s_0^d \rho^{d(d+1)/2}.$$

Proposition 1 (Crossover depth). *g is increasing over the regime of interest, so there is a unique smallest integer d^* with $g(d) > \tau$ for all $d \geq d^*$. Below d^* the plain similarity store is good enough and cheaper. At and beyond d^* the structured store wins on both accuracy and cost. The empirical statement “the graph helps iff $d \geq 2$ ” is the measured claim $d^* \approx 2$.*

Fusing into the metric: the rebuttal to “just buy more tokens”

With cost to correct $= c/P$ and $\eta = P/c$,

$$\frac{\eta_{\text{graph}}}{\eta_{\text{sim}}} = \underbrace{\left(\frac{q}{s_0}\right)^d \rho^{-d(d+1)/2}}_{\text{diverges in } d} \times \underbrace{\frac{c_{\text{sim}}}{c_{\text{graph}}}}_{\approx 1/2 \text{ (constant)}}.$$

The token penalty is a *constant* factor of about two. The recovery gap is *super exponential*. A constant cannot beat a super exponential, so $\eta_{\text{graph}}/\eta_{\text{sim}} \rightarrow \infty$ *even though the graph costs $2 \times$ per call*. We do not hide the 14K against 7K number. We show it cannot matter. This single line answers the entire “buy more tokens” objection.

Two heavy extensions (we include exactly these two)

(1) Tightness of the quadratic exponent. We argue the $d(d+1)/2$ penalty is not an artifact of the geometric ansatz but a property of any linkless retriever facing chain internal interference.

Claim 1 (Quadratic is unavoidable). *Under the interference model, any flat (linkless) retriever satisfies $\log P_{sim}(d) \leq -cd^2 + O(d)$ for some $c > 0$. Hence no similarity only method escapes a quadratic in d log penalty. The structured linear exponent is categorically, not incidentally, better.*

(2) Closed form optimal budget B^* . For similarity you can spend a token budget B widening K to fight interference, but precision falls as the window grows (context rot), so recovery $P(B)$ is increasing and concave while cost is linear. Maximizing $\eta(B) = P(B)/B$ gives the tangent from origin condition

$$P'(B^*) = \frac{P(B^*)}{B^*},$$

a single break even budget beyond which *adding tokens lowers η* . The graph sidesteps this by not paying for width at all. B^* is the quotable practitioner number, the point where buying more tokens starts hurting.

The single figure

Figure 1 is $P(d)$ against d for both regimes, two diverging curves, with the measured depth sweep overlaid on the predicted q^d and $s_0^d \rho^{d(d+1)/2}$, and η as the satellite result converting the gap into cost to correct. One core equation, one core figure.

6. What Is Locked, and What Remains

Locked. Several pieces are now settled. The idea is depth rather than length. The storyline puts capability first and efficiency second, with decisions as the hero. The construct is context retrieval quality, isolated. The datasets form three layers. The arms are the ladder together with the controls and the two isolating baselines. The metric is $\eta = P/c$, justified through the numerator. The math skeleton is one object f_k , the $s_0 \rho^k$ justification, q^d against $s_0^d \rho^{d(d+1)/2}$, the crossover d^* , the η fusion, and the two extensions. And the depth axis is the single dial that runs through the theory, the experiment, and every figure.

Remaining, before and during drafting. First, a registration document committed in advance, covering the claims, datasets, metrics, and model list, before any run. Second, the exact spec stripping recipe per dataset. Third, the empirical measurement of q , s_0 , and ρ on real depth labeled chains, which is also the test of the load bearing assumption. Fourth, the model list for the multi model robustness check. None of these block writing the theory. They are the experimental program the theory sets up.

Shallow tasks hide the problem. Deep tasks expose it. The paper is the depth at which the world tips over.